

Frontend Engineering Take-Home Assessment

Role Alignment: UI Front End Developer

Focus Areas: Scalable Frontend Engineering, Accessibility, Performance, Design Systems, Secure Coding, Collaboration

Submission: Public GitHub Repository + Deployment URL + README

This exercise is intended to simulate a production-grade frontend engineering assignment. We are assessing not only whether the application works, but also how you think as an engineer: architecture decisions, scalability, performance awareness, accessibility, maintainability, testing strategy, collaboration readiness, and attention to user experience. You are encouraged to use modern frontend tooling, AI-assisted development tools, component libraries, and industry-standard engineering practices.

Project Objective

Build a responsive banking frontend platform for Eagle Bank. The application should integrate with backend APIs and provide a modern, accessible, secure, and performant user experience.

For backend integration, you are not required to build a real backend service. Please use mock APIs using local JSON data. The application should support the following features:

1. Authentication

Implement a complete mocked authentication flow.

Required screens:

- Login page
- Registration page

Requirements:

- Store authentication state on the frontend
- Protect authenticated routes
- Persist session using local storage or cookies
- Display validation and error states
- Include loading states during login/register actions

Mock endpoints:

- POST /api/auth/register
- POST /api/auth/login
- POST /api/auth/logout
- GET /api/auth/me

2. Dashboard

Create a dashboard page after login.

The dashboard should display:

- Welcome message with user name
- Total account balance
- Summary cards:
 - Current balance
 - Monthly deposits
 - Monthly withdrawals
 - Recent transactions
- Quick actions section

Include:

- Empty states
- Responsive layout

Mock endpoint:

- GET /api/dashboard

3. Accounts Management

Users should be able to view their bank accounts.

Requirements:

- Display multiple account types:
 - Savings
 - Credit
- Show:
 - Account number
 - Available balance
 - Account type
 - Status
- Support responsive table or card layouts

Mock endpoints:

- GET /api/accounts
- GET /api/accounts/:id

4. Transactions

Implement transaction history management.

Requirements:

- Display transaction list
- Support:
 - Filter by date range
 - Sorting by amount/date
- Include pagination or infinite scrolling
- Show transaction details

Transaction types:

- Deposit
- Withdrawal
- Transfer

Mock endpoints:

- GET /api/transactions

- GET /api/transactions/:id

5. User Profile

Implement profile management functionality.

Requirements:

- View profile information
- Edit:
 - Full name
 - Email
 - Phone number
 - Address
- Upload avatar (frontend only/mock)
- Form validation

Mock endpoints:

- GET /api/profile
- PUT /api/profile

6. Error Handling

The application should gracefully handle failures.

Requirements:

- API error states
- Empty states
- Loading indicators
- Retry actions
- 404 page
- Generic error boundary/fallback UI

Technical Expectations

Preferred Stack:

- React + TypeScript (preferred)
- Next.js / Angular / Vue accepted

The implementation should demonstrate:

- Component-driven architecture
- Responsive & accessible UI design
- Frontend performance optimisation
- Backend integration knowledge
- Reusable and scalable UI patterns
- Secure frontend coding practices
- Error handling and resilience
- Clean code structure and maintainability

Design System & UI Engineering

We highly value frontend engineers who can build systems, not just pages.

Please demonstrate:

- Consistent typography and spacing system
- Reusable buttons, forms, modals, and tables
- Shared theme/token strategy if applicable
- Accessible components and semantic HTML
- Strong visual hierarchy

Bonus:

- Design system documentation
- Storybook integration

Performance & Optimisation

Performance is an important part of this assessment.

- Lazy loading and route splitting
- Optimised rendering patterns
- Efficient state management
- Avoiding unnecessary re-renders
- Lighthouse best practices

Accessibility Requirement

Your implementation should demonstrate accessibility awareness.

- Keyboard navigation
- Semantic HTML
- Proper labels and ARIA usage
- Contrast and readability
- Focus management
- Screen reader compatibility basics

Animations & Micro-Interactions

We value polished user experiences. Include subtle but meaningful interactions where appropriate:

- Smooth transitions
- Interactive hover/focus states
- Loading animations/skeletons
- Motion that enhances usability rather than distracts

Engineering & Collaboration Standards

Your submission should reflect professional engineering practices:

- Meaningful Git commits
- Clean pull-request style code structure

- Code readability and maintainability
- Thoughtful abstraction levels
- Proper separation of concerns
- Clear documentation

Testing Expectations

Include meaningful unit tests covering critical functionality.

Suggested areas:

- Authentication flows
- Form validation
- API integrations

README Requirements

Your README should explain:

- Architecture decisions
- Folder structure rationale
- State management approach
- Performance considerations
- Accessibility considerations
- How to run and test the application

Evaluation Criteria

We will evaluate:

- Frontend architecture quality
- Scalability and maintainability
- UX/UI quality
- Accessibility awareness
- Performance optimisation
- Testing strategy
- Engineering communication

Evaluation Breakdown

Category	Weight
Frontend Architecture	20%
Responsive & Accessible Design	15%
Performance Optimisation	15%
Design System Thinking	10%
Backend Integration	10%
Testing Quality	15%
Code Maintainability	10%
Documentation & Communication	5%

We are looking for engineers who can balance technical excellence, business understanding, maintainability, user experience, and collaboration. Your submission should reflect production-quality frontend engineering standards.